You're a guest in Henrique Cunha's Notion. Get full access to Notion in your own workspace. Set up workspace

Projeto Hands-on — RTL Design Track: Acelerador AES com interface SPI e baixo consumo

Visão geral

O *Advanced Encryption Standard*, ou AES, é um padrão de criptografia simétrica amplamente utilizado para proteger dados. Neste projeto, os alunos da trilha RTL desenvolverão um sistema de topo com datapath AES, a mesma arquitetura (*AES Top-Level System*) adotada no projeto integrador da trilha de Design Verification. O sistema é configurado e controlado por uma interface SPI, recebe e devolve dados por uma interface de dados com memória e conta com blocos de reset e de geração de clock.

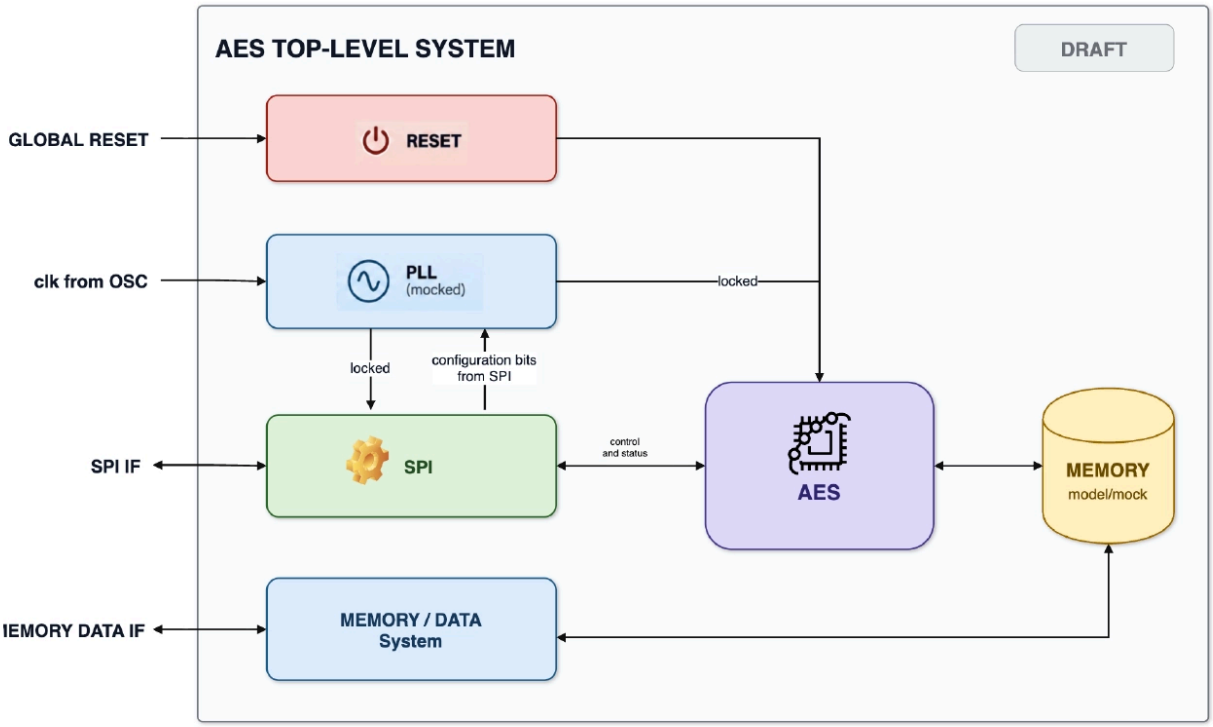


Figura 1 – Arquitetura-base do sistema de topo, a mesma utilizada pela trilha de Design Verification.

O projeto percorre o fluxo típico de desenvolvimento de um IP digital: do RTL à primeira síntese, passando pela verificação formal de equivalência e pela aplicação de técnicas de baixo consumo, com a intenção de potência descrita em UPF. Ao final, cada aluno terá um IP sintetizável, com equivalência formal entre RTL e netlist comprovada, verificado em simulação por um aluno da trilha de verificação e preparado para operar em diferentes modos de energia.

O projeto terá duração de **15 semanas**, com **um entregável por semana**.

> **Escopo deste documento:** descrever como o projeto será executado (etapas, cronograma, entregáveis e critérios de aceite). A especificação funcional de cada módulo será disponibilizada separadamente pelo instrutor.

Objetivos de aprendizagem

Ao concluir o projeto, o aluno deverá ser capaz de:

- transformar uma especificação em arquitetura, microarquitetura e RTL sintetizável;
- integrar um sistema de topo com reset, geração de clock, configuração por SPI, datapath e memória;
- integrar blocos com domínios de clock distintos, tratando a sincronização entre eles;
- preparar restrições de timing e executar uma síntese lógica, interpretando os relatórios de área, timing e potência;
- utilizar verificação formal de equivalência entre RTL e netlist com o Formality;
- descrever a intenção de potência de um projeto com UPF;
- aplicar e avaliar técnicas de baixo consumo, como clock gating, power gating, isolamento de operandos e retenção de estado;
- documentar e apresentar tecnicamente as decisões de projeto.

Organização

A organização seguirá o fluxo **cliente/instrutor → alunos**.

- **Instrutor:** define o escopo, a especificação funcional e os critérios de aceite; fornece as ferramentas e o PDK; acompanha as entregas semanais; conduz as defesas e revisões técnicas.
- **Alunos:** cada aluno prepara o seu ambiente inicial (repositório e scripts), gerencia o seu backlog, desenvolve o seu próprio IP, executa as atividades de projeto, registra as decisões e produz os entregáveis.
- **Verificador parceiro:** cada aluno da trilha RTL terá o seu IP verificado por um aluno da trilha de verificação (ver seção *Interação com a trilha de verificação*).

Ritmo semanal

Momento	Atividade
Início da semana	Planejamento: atualização do backlog pelo aluno e definição das tarefas da semana
Meio da semana	Ponto de acompanhamento com o instrutor (bloqueios e dúvidas)
Fim da semana	Entrega do artefato da semana no repositório e revisão com o instrutor com sabatina oral a respeito do que foi feito

Regras gerais para os entregáveis

- Todo entregável deve estar no repositório Git do aluno, em uma *tag* ou *branch* identificada pela semana (ex.: `w06-rtl-v1.0`).
- Todo entregável deve vir acompanhado de um relatório curto (`docs/reports/semana_XX.md`) com o que foi feito, os resultados obtidos, os problemas encontrados e os próximos passos.
- Relatórios de ferramentas (lint, síntese, formal, UPF) devem ser versionados junto com os scripts que os geraram.
- Uma entrega só é considerada concluída quando atende ao critério de aceite da semana.
- Defeitos reportados pelo verificador parceiro são registrados como *issues* no repositório e tratados no backlog do aluno.

Fases do projeto

Fase	Semanas	Foco
1. Especificação e arquitetura	1–2	Estudo, ambiente e definição da arquitetura
2. Desenvolvimento do RTL	3–6	AES, SPI, RESET, subsistema de dados, modelos do PLL e da memória e integração
3. Síntese inicial	7	Restrições, síntese e baseline de área, timing e potência
4. Verificação formal de equivalência	8–9	Equivalência RTL × netlist com o Formality
5. Intenção de potência	10–11	Arquitetura de energia e UPF
6. Técnicas de baixo consumo	12–14	Power gating, clock gating, retenção e comparação com o baseline
7. Encerramento	15	Documentação, apresentação final e artigo

Cronograma semanal

Semana 1 — Kick-off, estudo e ambiente

Atividades

- Apresentação do projeto e da especificação funcional.
- Estudo do algoritmo AES (FIPS-197) e do protocolo SPI (modos de operação, polaridade e fase de clock).
- Estudo da arquitetura do sistema de topo (*AES Top-Level System*), comum às trilhas de RTL e de verificação.
- Preparação do ambiente inicial pelo aluno: repositório, scripts de execução e configuração das ferramentas fornecidas (simulador, lint).
- Criação do backlog inicial do projeto.

Entregável: repositório estruturado, backlog inicial, ambiente executando um exemplo mínimo (compilação, lint e simulação) e relatório de estudo sobre AES e SPI.

Critério de aceite: o repositório pode ser clonado em um ambiente limpo e o fluxo mínimo executado com um único comando.

Semana 2 — Arquitetura e microarquitetura

Atividades

- Definição do diagrama de blocos e das interfaces internas entre os módulos, a partir da arquitetura do sistema de topo.
- Detalhamento das interfaces de topo (GLOBAL RESET, clk from OSC, SPI IF e MEMORY DATA IF) conforme a especificação do sistema de topo.
- Definição do mapa de registradores.
- Definição da estratégia de clock e reset (bloco de RESET, PLL mockado e sinal *locked*). O SPI opera nativamente no SCLK: registrador de deslocamento, contador de bits e saída MISO no domínio do SCLK; banco de registradores e interface com o AES no domínio do sistema. O PLL sai do reset com uma configuração padrão e trava sozinho; os seus registradores de configuração ficam no banco de registradores, no domínio do sistema, e só são reprogramados pelo SPI depois do *locked*. A estratégia deve definir a travessia entre os dois domínios e o reset de cada um.
- Definição da interface dos modelos comportamentais do PLL e da memória (nome do módulo, portas, larguras e significado dos bits de configuração), compatível com a dos modelos utilizados pela trilha de verificação.
- Escolha da microarquitetura do AES (por exemplo, iterativa por rodada ou outra alternativa justificada).
- Definição da interface do núcleo AES (controle e status vindos do SPI, chave e acesso à memória de dados), contra a qual o núcleo será desenvolvido antes do SPI.
- Apresentação da arquitetura e das interfaces de topo ao verificador parceiro.

Entregável: documento de arquitetura (v1.0), com diagramas, interfaces de topo e internas, interfaces dos modelos do PLL e da memória, mapa de registradores e justificativa das decisões, apresentado em uma defesa curta.

Critério de aceite: arquitetura aprovada pelo instrutor na defesa.

Semana 3 — RTL do núcleo AES I: datapath das rodadas

Atividades

- Implementação das transformações de rodada do AES (SubBytes, ShiftRows, MixColumns e AddRoundKey).
- Testes unitários de cada transformação com valores intermediários de referência do padrão FIPS-197.

Entregável: RTL do datapath de rodadas e testes unitários das transformações.

Critério de aceite: todas as transformações conferem com os valores intermediários de referência; lint sem erros.

Semana 4 — RTL do núcleo AES II: expansão de chave e controle

Atividades

- Implementação da expansão de chave.
- Implementação do controle do núcleo (sequenciamento das rodadas, início e fim de operação), seguindo a interface definida na semana 2.
- Testbench dirigido com os vetores de teste do padrão FIPS-197.

Entregável: RTL completo do núcleo AES e testbench com os vetores do padrão.

Critério de aceite: todos os vetores de teste de referência passam em simulação; lint sem erros.

Semana 5 — RTL do SPI e do RESET e modelo do PLL

Atividades

- Implementação da interface SPI.
- Implementação do estágio de sincronização entre o domínio do SCLK e o domínio do sistema.
- Implementação do banco de registradores (configuração, controle e status, incluindo os bits de configuração do PLL).
- Implementação do bloco de RESET.
- Implementação de um modelo comportamental simples do PLL para os testes iniciais (configuração padrão após o reset, geração do clock do sistema, sinal *locked* e resposta à reconfiguração), seguindo a interface definida na semana 2. O modelo fica em `models/`, fora do RTL sintetizável.
- Testbench dirigido para escrita e leitura de registradores via SPI, sequência de reset e travamento do PLL.

Entregável: RTL do SPI com banco de registradores e do RESET, modelo simples do PLL, testbench básico e relatório de lint.

Critério de aceite: escrita e leitura de todos os registradores via SPI funcionando em simulação; sequência de reset e *locked* conforme a especificação; lint sem erros e com avisos justificados. A análise de CDC é feita na semana 6, na integração.

Semana 6 — Subsistema de dados, integração e congelamento do RTL v1.0

Atividades

- Implementação do MEMORY / DATA System.
- Implementação de um modelo comportamental simples da memória (*mock*) para os testes iniciais, seguindo a interface definida na semana 2, e sua integração.
- Integração do top-level conforme a arquitetura do sistema de topo.
- Teste de ponta a ponta: configuração via SPI → entrada pela MEMORY DATA IF → AES → saída pela MEMORY DATA IF.
- Verificação de CDC (travessia entre os domínios do SCLK e do sistema).

- Correção dos problemas encontrados.
- Entrega do RTL v1.0 ao verificador parceiro.

Entregável: top-level integrado, teste de ponta a ponta, relatórios de lint e CDC e *tag* **RTL v1.0**.

Critério de aceite: fluxo completo configuração → entrada → AES → saída funcionando em simulação; interfaces de topo conforme a especificação do sistema de topo; nenhuma violação de CDC sem justificativa. O RTL v1.0 passa a ser a referência (baseline) para as fases seguintes.

Semana 7 — Rodada inicial de síntese

Atividades

- Escrita das restrições de timing (SDC): clocks (o clock gerado pelo PLL e o SCLK, declarados como assíncronos entre si), entradas, saídas e restrições dos caminhos de sincronização.
- Tratamento dos modelos comportamentais (PLL mockado e memória) como caixas-pretas ou macros, conforme orientação do instrutor.
- Síntese lógica do RTL v1.0 com a biblioteca de células do PDK fornecido pelo instrutor.
- Análise dos relatórios de área, timing e potência.
- Identificação de caminhos críticos e de construções não sintetizáveis ou ineficientes.

Entregável: scripts de síntese, arquivo SDC, netlist e relatório de baseline (área, timing e potência estimada).

Critério de aceite: síntese sem erros, sem latches não intencionais e com timing atendido na frequência alvo, ou com o plano de correção documentado.

Semana 8 — Verificação de equivalência I: RTL × netlist

Atividades

- Preparação do fluxo de equivalência no Formality: leitura do RTL (referência), da netlist sintetizada (implementação), das bibliotecas do PDK e do arquivo de *guidance* (SVF) gerado pela síntese.
- Tratamento dos modelos comportamentais (PLL e memória) como caixas-pretas, de forma consistente com a síntese.
- Execução das etapas de *match* e *verify* e análise dos pontos não casados, não verificados ou não equivalentes.

Entregável: scripts do Formality e relatório de equivalência entre o RTL v1.0 e a netlist do baseline.

Critério de aceite: fluxo executável por script; todos os pontos de comparação casados ou com justificativa; resultado da verificação registrado.

Semana 9 — Verificação de equivalência II: depuração e fechamento

Atividades

- Depuração das não equivalências e dos pontos não verificados (por exemplo, otimizações da síntese sem *guidance*, registradores removidos ou constantes).

- Exercício de diagnóstico: inserção intencional de uma alteração no RTL ou na netlist para observar como o Formality aponta a diferença.
- Correção dos defeitos encontrados e dos defeitos reportados pelo verificador parceiro até o momento, com nova síntese e nova verificação de equivalência.

Entregável: relatório consolidado de equivalência, registro das não equivalências e de suas causas e *tag* **RTL v1.1**.

Critério de aceite: equivalência entre o RTL v1.1 e a sua netlist confirmada pelo Formality, sem pontos não verificados sem justificativa.

Semana 10 — Arquitetura de energia

Atividades

- Análise do perfil de atividade do projeto: quais blocos ficam ociosos e em quais situações.
- Definição dos modos de operação (por exemplo, ativo, ocioso e desligado).
- Definição dos domínios de potência e da tabela de estados de potência (*power state table*), identificando os blocos que precisam permanecer sempre ligados (por exemplo, RESET, PLL e SPI).
- Definição das estratégias de isolamento e retenção: quais sinais isolar e quais registradores reter.
- Definição da sequência de desligamento e religamento.

Entregável: documento de arquitetura de energia, com domínios, modos, tabela de estados e sequências, apresentado em uma defesa curta.

Critério de aceite: arquitetura de energia aprovada pelo instrutor.

Semana 11 — Intenção de potência com UPF

Atividades

- Escrita do UPF: domínios de potência, redes de alimentação, chaves de potência, estratégias de isolamento e retenção e, se aplicável, level shifters.
- Verificação estática do UPF (consistência entre UPF e RTL).
- Simulação *power-aware* dos modos de operação.

Entregável: arquivo UPF, relatório de verificação estática e relatório de simulação *power-aware*.

Critério de aceite: UPF sem erros na verificação estática; simulação mostrando corrupção de sinais em domínios desligados e comportamento correto do isolamento e da retenção.

Semana 12 — Técnicas de baixo consumo I: power gating

Atividades

- Implementação do controlador de energia e da sequência de power gating (isolar, salvar, desligar, religar, restaurar, liberar).

- Integração do controlador com o banco de registradores (comandos e status dos modos de energia).
- Atualização do testbench para exercitar as transições entre modos de energia.

Entregável: RTL com o controlador de energia, testes das transições entre modos e relatório de simulação *power-aware*.

Critério de aceite: sequências de desligamento e religamento funcionando em simulação *power-aware*, com isolamento e retenção corretos.

Semana 13 — Técnicas de baixo consumo II: clock gating e outras técnicas

Atividades

- Clock gating: identificação de oportunidades arquiteturais e uso de células de clock gating integradas (ICG).
- Outras técnicas, conforme a arquitetura aprovada: isolamento de operandos, redução de atividade de chaveamento e desligamento de lógica ociosa.
- Entrega do RTL v2.0 ao verificador parceiro para regressão.

Entregável: *tag* **RTL v2.0 (low power)**, testes atualizados e relatório descrevendo cada técnica aplicada e onde ela foi aplicada.

Critério de aceite: funcionalidade da v1.0 preservada; transições entre modos de energia funcionando em simulação *power-aware*.

Semana 14 — Síntese low power e comparação com o baseline

Atividades

- Síntese do RTL v2.0 com o UPF, incluindo a inserção automática de clock gating.
- Comparação com o baseline da semana 7: área, timing e potência, por modo de operação.
- Reexecução da verificação de equivalência no Formality entre o RTL v2.0 e a netlist low power, considerando a intenção de potência (UPF) e o clock gating inserido na síntese.

Entregável: relatório comparativo baseline × low power, com tabelas e análise crítica dos ganhos e custos de cada técnica, e relatório de equivalência.

Critério de aceite: síntese com UPF sem erros; ganhos e custos quantificados e justificados; equivalência entre o RTL v2.0 e a netlist low power confirmada.

Semana 15 — Encerramento

Atividades

- Consolidação da documentação técnica (arquitetura, arquitetura de energia, guia de uso do repositório).
- Consolidação dos defeitos reportados pelo verificador parceiro e do seu estado final.
- Preparação e realização da apresentação final.
- Elaboração do artigo técnico ou científico.

- Organização final do repositório.

Entregável: *release* final no repositório, documentação completa, apresentação final e versão preliminar do artigo.

Critério de aceite: projeto reproduzível a partir do repositório (simulação, síntese, formal e UPF executáveis pelos scripts); apresentação realizada.

Resumo dos entregáveis

Semana	Entregável	Marco
1	Ambiente configurado + backlog inicial + relatório de estudo	Kick-off
2	Documento de arquitetura v1.0, com as interfaces de topo	Defesa da arquitetura; apresentação ao verificador parceiro
3	RTL do datapath de rodadas do AES	
4	RTL completo do núcleo AES	
5	RTL SPI + registradores + RESET + modelo do PLL	
6	Subsistema de dados + top-level integrado + CDC	**RTL v1.0**; entrega ao verificador parceiro
7	Síntese inicial + relatório de baseline	Baseline
8	Scripts do Formality + relatório de equivalência do baseline	
9	Relatório consolidado de equivalência	**RTL v1.1**
10	Documento de arquitetura de energia	Defesa da arquitetura de energia
11	UPF + verificação estática + simulação power-aware	
12	RTL com controlador de energia e power gating	
13	RTL com clock gating e demais técnicas de baixo consumo	**RTL v2.0**; entrega ao verificador parceiro
14	Síntese low power + relatório comparativo	
15	Release final, documentação, apresentação e artigo	Apresentação final

---

Estrutura sugerida do repositório

```
. ├── README.md ├── docs/ │ ├── spec/ # especificação funcional (fornecida) │ ├── ar
chitecture/ # arquitetura e arquitetura de energia │ └── reports/ # relatórios seman
ais ├── rtl/ # código RTL ├── models/ # modelos simples do PLL e da memória (testes
iniciais) ├── tb/ # testbenches ├── formal/ # scripts e relatórios do Formality ├──
syn/ # SDC, scripts e relatórios de síntese ├── upf/ # arquivos UPF e relatórios └──
scripts/ # automação do fluxo
```

Entregas finais

Ao final do projeto, cada aluno deverá apresentar:

- documento de arquitetura e documento de arquitetura de energia;
- código RTL (v1.0, v1.1 e v2.0 identificados por *tags*);
- testbenches e vetores de teste;
- restrições de timing e scripts de síntese;
- relatórios de síntese (baseline e low power);
- scripts e relatórios de verificação de equivalência;
- arquivo UPF e relatórios de verificação *power-aware*;
- relatório comparativo das técnicas de baixo consumo;
- histórico dos defeitos reportados pelo verificador parceiro e das correções;
- repositório Git organizado e reproduzível;
- apresentação final;
- artigo preparado para submissão (caso possível).

Interação com a trilha de verificação

Cada aluno da trilha RTL formará uma dupla com um aluno da trilha de *Design Verification*. O verificador parceiro será responsável por verificar em simulação o IP desenvolvido pelo aluno de RTL, enquanto o aluno de RTL mantém a responsabilidade pelo projeto, pela síntese, pela verificação de equivalência e pelas técnicas de baixo consumo.

As duas trilhas trabalham sobre a mesma arquitetura de sistema de topo e a mesma especificação. Na trilha de verificação, cada aluno constrói VIPs para as quatro interfaces de topo (SPI, CLOCK, RESET e MEMORY DATA), um modelo de referência para o AES e um ambiente que verifica o fluxo fim a fim.

A verificação acontece em duas etapas:

1. **RTL de referência.** Para não depender do andamento desta trilha, a trilha de verificação desenvolve o seu ambiente sobre um RTL pronto, fornecido pelo instrutor de verificação.
2. **RTL do aluno parceiro.** Em um segundo momento, o ambiente passa a ser executado sobre o RTL desenvolvido pelo aluno de RTL, para encontrar defeitos tanto no RTL quanto no próprio ambiente de verificação.

Para que o ambiente construído na primeira etapa possa ser reaproveitado na segunda, as interfaces de topo do RTL desta trilha (nomes dos sinais, protocolos e temporizações) devem ser compatíveis com as do RTL de referência.

O mesmo vale para os modelos comportamentais do PLL e da memória. Cada aluno de RTL escreve modelos simples para os seus testes iniciais, para não depender da trilha de verificação. Na segunda etapa, esses modelos são substituídos pelos da trilha de verificação, que têm a mesma interface. Diferenças de comportamento entre os dois modelos são registradas como *issues*, pois indicam defeito no RTL, defeito em um dos modelos ou ambiguidade da especificação.

A interação seguirá três pontos de contato ao longo do projeto:

Semana	Ponto de contato	O que é compartilhado
2	Apresentação da arquitetura	Documento de arquitetura, interfaces de topo e mapa de registradores, para que o verificador parceiro identifique diferenças em relação ao RTL de referência
6	Primeira entrega do IP	*Tag* RTL v1.0, a partir da qual o ambiente de verificação passa a ser executado sobre o RTL do aluno parceiro
13	Segunda entrega do IP	*Tag* RTL v2.0, com os modos de energia, para regressão

Regras da interação:

- O verificador parceiro acessa o IP pela *tag* entregue, e não pelo estado corrente do repositório.
- Os defeitos encontrados são registrados como *issues* no repositório do aluno de RTL, com descrição, cenário de reprodução e evidência (log ou forma de onda).
- O aluno de RTL analisa cada *issue* e a classifica como defeito do RTL, defeito do ambiente de verificação ou ambiguidade da especificação; ambiguidades são levadas ao instrutor.
- As correções são entregues em novas *tags* e comunicadas ao verificador parceiro.
- O escopo de verificação da trilha de verificação é funcional e não inclui os modos de energia. A regressão sobre a v2.0 confirma que a funcionalidade não foi alterada; os modos de energia são verificados pelo próprio aluno de RTL (simulação *power-aware*).
- A apresentação final de cada aluno deve incluir o resultado da verificação feita pelo verificador parceiro.